

Ch7. Gradient Descent

Section 7.2. Gradient Descent Optimization

第 7 章-梯度下降 – 第 7.2 节-梯度下降优化

《深度学习基础》课程小组

2026 年 8 月 1 日

1 P1

There is little hope of finding an analytical solution to the equation $\nabla E(\mathbf{w}) = 0$ for an error function as complex as one defined by a neural network, and so we resort to iterative numerical procedures.

The optimization of continuous nonlinear functions is a widely studied problem, and there exists an extensive literature on how to solve it efficiently.

Most techniques involve choosing some initial value $\mathbf{w}^{(0)}$ for the weight vector and then moving through weight space in a succession of steps of the form

$$\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} + \Delta \mathbf{w}^{(\tau-1)} \quad (7.15)$$

where τ labels the iteration step.

Different algorithms involve different choices for the weight vector update $\Delta \mathbf{w}^{(\tau)}$.

翻译:

对于像神经网络所定义的误差函数这样复杂的函数，几乎不可能找到方程 $\nabla E(\mathbf{w}) = 0$ 的解析解，因此我们不得不求助于迭代数值方法。

连续非线性函数的优化是一个被广泛研究的问题，关于如何高效地求解此类问题，已有大量文献可供参考。

大多数方法都需要为权重向量选取某个初始值 $\mathbf{w}^{(0)}$ ，然后在权重空间中通过一系列如下形式的步骤逐步移动：

$$\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} + \Delta \mathbf{w}^{(\tau-1)}$$

其中 τ 表示迭代步数。

不同的算法对应着权重向量更新量 $\Delta \mathbf{w}^{(\tau)}$ 的不同选择。

2 P2

Because of the complex shape of the error surface for all but the simplest neural networks, the solution found will depend, among other things, on the particular choice of initial parameter values $\mathbf{w}^{(0)}$.

To find a sufficiently good solution, it may be necessary to run a gradient-based algorithm multiple times, each time using a different randomly chosen starting point, and comparing the resulting performance on an independent validation set.

翻译：

由于除最简单的神经网络之外，所有神经网络的误差曲面形状都极为复杂，因此所找到的解将取决于（除其他因素外）初始参数值 $\mathbf{w}^{(0)}$ 的特定选取。

为了找到一个足够好的解，可能需要多次运行基于梯度的优化算法，每次使用不同的随机初始点，并在一个独立的验证集上比较所得结果的性能表现。

3 P3

The gradient of an error function for a deep neural network can be evaluated efficiently using the technique of error backpropagation, and applying this gradient information can lead to significant improvements in the speed of network training. We can see why this is so, as follows.

翻译：

深度神经网络误差函数的梯度可以利用误差反向传播技术进行高效计算，而利用这些梯度信息能够显著提升网络的训练速度。其原因如下所述。

4 P4

In the quadratic approximation to the error function given by (7.3), the error surface is specified by the quantities $\mathbf{b}$ and $\mathbf{H}$, which contain a total of $W(W+3)/2$ independent elements (because the matrix $\mathbf{H}$ is symmetric), where W is the dimensionality of $\mathbf{w}$ (i.e., the total number of learnable parameters in the network).

The location of the minimum of this quadratic approximation therefore depends on $\mathcal{O}(W^2)$ parameters, and we should not expect to be able to locate the minimum until we have gathered $\mathcal{O}(W^2)$ independent pieces of information.

If we do not make use of gradient information, we would expect to have to perform $\mathcal{O}(W^2)$ function evaluations, each of which would require $\mathcal{O}(W)$ steps. Thus, the computational effort needed to find the minimum using such an approach would be $\mathcal{O}(W^3)$.

翻译：

在由式 (7.3) 给出的误差函数的二次近似中，误差曲面由量 $\mathbf{b}$ 和 $\mathbf{H}$ 所确定，它们共包含 $W(W+3)/2$ 个独立元素（因为矩阵 $\mathbf{H}$ 是对称的），其中 W 是 $\mathbf{w}$ 的维数（即网络中可学习参数的总数）。

因此，该二次近似极小值点的位置取决于 $\mathcal{O}(W^2)$ 个参数，我们不应期望在收集到 $\mathcal{O}(W^2)$ 个独立信息之前就能定位到该极小值点。

如果不利用梯度信息，我们预计需要进行 $\mathcal{O}(W^2)$ 次函数求值，而每次求值都需要 $\mathcal{O}(W)$ 步运算。因此，使用这种方法寻找极小值所需的计算量为 $\mathcal{O}(W^3)$ 。

问：为什么在神经网络优化中必须使用梯度信息？

答：其推理逻辑如下：

1. 问题的规模

误差函数的二次近似由梯度向量 $\mathbf{b}$ （ W 个分量）和 Hessian 矩阵 $\mathbf{H}$ （对称矩阵， $W(W+1)/2$ 个独立

元素) 完全确定, 合计 $W(W+3)/2$ 个参数。因此, 要确定极小值点的位置, 本质上需要获取 $\mathcal{O}(W^2)$ 个独立信息。

2. 不用梯度的代价

如果不使用梯度信息, 每次函数求值只能获得 1 个标量信息 (即该点的函数值)。要凑够 $\mathcal{O}(W^2)$ 个独立信息, 就需要 $\mathcal{O}(W^2)$ 次求值。而每次求值本身需要 $\mathcal{O}(W)$ 次运算 (前向传播), 所以总计算量为:

$$\mathcal{O}(W^2) \times \mathcal{O}(W) = \mathcal{O}(W^3)$$

3. 言外之意 (与下文衔接)

这段话实际上是在为反向传播做铺垫。如果使用梯度信息, 每次计算梯度就能一次性获得 $\mathcal{O}(W)$ 个独立信息 (因为梯度是一个 W 维向量), 那么只需 $\mathcal{O}(W)$ 次迭代即可收集到足够的信息, 总计算量降为 $\mathcal{O}(W^2)$ ——比不用梯度的方法快了一个数量级。

总结: 对于拥有大量参数的神经网络, 不使用梯度信息的优化方法在计算复杂度上是不可接受的, 这正是反向传播技术至关重要的根本原因。

5 P5

Now compare this with an algorithm that makes use of the gradient information.

Because ∇E is a vector of length W , each evaluation of ∇E brings W pieces of information, and so we might hope to find the minimum of the function in $\mathcal{O}(W)$ gradient evaluations.

As we shall see, by using error backpropagation, each such evaluation takes only $\mathcal{O}(W)$ steps and so the minimum can now be found in $\mathcal{O}(W^2)$ steps.

Although the quadratic approximation only holds in the neighbourhood of a minimum, the efficiency gains are generic.

For this reason, the use of gradient information forms the basis of all practical algorithms for training neural networks.

翻译：

现在将上述情况与利用梯度信息的算法进行比较。

由于 ∇E 是一个长度为 W 的向量，每次计算 ∇E 就能带来 W 个信息，因此我们可以期望通过 $\mathcal{O}(W)$ 次梯度计算就找到函数的极小值点。

正如我们将要看到的，利用误差反向传播，每次梯度计算仅需 $\mathcal{O}(W)$ 步运算，因此极小值点现在可以在 $\mathcal{O}(W^2)$ 步内找到。

尽管二次近似仅在极小值点附近成立，但这种效率上的提升是具有普遍意义的。

正是基于这一原因，梯度信息的使用构成了所有实用神经网络训练算法的基础。

解释：这段话与上一段形成对比论证，核心结论是：

	不用梯度	用梯度（反向传播）
每次计算获得的信息量	1 个（标量函数值）	W 个（梯度向量）
所需计算次数	$\mathcal{O}(W^2)$	$\mathcal{O}(W)$
每次计算的代价	$\mathcal{O}(W)$	$\mathcal{O}(W)$
总计算量	$\mathcal{O}(W^3)$	$\mathcal{O}(W^2)$

关键点：

1. 信息效率的差异：梯度是一个 W 维向量，一次计算相当于同时获取了 W 个独立信息，比单纯求函数值高效了 W 倍。
2. 反向传播的关键作用：反向传播使得计算整个梯度向量的代价仅为 $\mathcal{O}(W)$ （与一次前向传播同阶），而非朴素的 $\mathcal{O}(W^2)$ （对每个参数分别求偏导）。这是效率提升的技术保障。
3. 局部论证，全局适用：作者坦承二次近似只在极小值点邻域内严格成立，但指出梯度方法带来的效率优势在实际中是普遍存在的，并非仅限于局部。
4. 最终结论：这解释了为什么所有实用的神经网络训练算法（SGD、Adam、L-BFGS 等）都以梯度计算为核心——这不是经验选择，而是计算复杂度上的必然要求。

6 P6

The simplest approach to using gradient information is to choose the weight update in (7.15) such that there is a small step in the direction of the negative gradient, so that

$$\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - \eta \nabla E(\mathbf{w}^{(\tau-1)}) \quad (7.16)$$

where the parameter $\eta > 0$ is known as the *learning rate*.

After each such update, the gradient is re-evaluated for the new weight vector $\mathbf{w}^{(\tau+1)}$ and the process repeated.

At each step, the weight vector is moved in the direction of the greatest rate of decrease of the error function, and so this approach is known as *gradient descent* or *steepest descent*.

Note that the error function is defined with respect to a training set, and so to evaluate ∇E , each step requires that the entire training set be processed.

Techniques that use the whole data set at once are called *batch methods*.

翻译:

利用梯度信息的最简单方法，是在公式 (7.15) 中令权重更新量沿负梯度方向取一小步，即

$$\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - \eta \nabla E(\mathbf{w}^{(\tau-1)})$$

其中参数 $\eta > 0$ 称为学习率 (learning rate)。

每次更新之后，需要对新的权重向量 $\mathbf{w}^{(\tau+1)}$ 重新计算梯度，然后重复上述过程。

在每一步中，权重向量都沿着误差函数下降最快的方向移动，因此这种方法被称为梯度下降法 (gradient descent) 或最速下降法 (steepest descent)。

需要注意的是，误差函数是定义在整个训练集上的，因此要计算 ∇E ，每一步都需要对整个训练集进行处理。

一次使用全部数据的技术称为批方法 (batch methods)。

几个要点提示:

原文术语	译文	备注
learning rate	学习率	控制每步”走多远”
gradient descent steepest descent	梯度下降 最速下降	两者同义
batch methods	批方法	与之对应的是”随机/小批量方法” 每次只用一个或一小批样本

7 P7

Deep learning methods benefit greatly from very large data sets.

However, batch methods can become extremely inefficient if there are many data points in the training set because each error function or gradient evaluation requires the entire data set to be processed.

To find a more efficient approach, note that error functions based on maximum likelihood for a set of independent observations comprise a sum of terms, one for each data point:

$$E(\mathbf{w}) = \sum_{n=1}^N E_n(\mathbf{w}). \quad (7.17)$$

翻译：

深度学习方法极大地受益于超大规模的数据集。

然而，如果训练集中包含大量数据点，批方法（batch methods）可能会变得极其低效，因为每次评估误差函数或梯度都需要对整个数据集进行处理。

为了寻找一种更高效的方法，我们注意到，对于一组独立观测数据，基于最大似然的误差函数是由若干项之和构成的，每一项对应一个数据点：

$$E(\mathbf{w}) = \sum_{n=1}^N E_n(\mathbf{w}).$$

8 P8

The most widely used training algorithms for large data sets are based on a sequential version of gradient descent known as *stochastic gradient descent* (Bottou, 2010), or SGD, which updates the weight vector based on one data point at a time, so that

$$\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - \eta \nabla E_n(\mathbf{w}^{(\tau-1)}). \quad (7.18)$$

This update is repeated by cycling through the data.

A complete pass through the whole training set is known as a *training epoch*.

This technique is also known as *online gradient descent*, especially if the data arises from a continuous stream of new data points.

Stochastic gradient descent is summarized in Algorithm 7.1.

翻译:

针对大规模数据集,最广泛使用的训练算法是基于梯度下降的顺序版本,被称为随机梯度下降(stochastic gradient descent, SGD) (Bottou, 2010)。该方法每次仅基于一个数据点来更新权重向量,即:

$$\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - \eta \nabla E_n(\mathbf{w}^{(\tau-1)}).$$

通过循环遍历数据集来重复执行这一更新操作。

完整地遍历一次整个训练集被称为一个训练轮次(training epoch)。

这种技术也被称为在线梯度下降(online gradient descent),尤其当数据来源于连续的新数据流时。

随机梯度下降法在算法 7.1 中进行了总结。

英文原文	中文译文	含义
batch methods	批方法	每次更新需使用全部数据
stochastic gradient descent (SGD)	随机梯度下降	每次更新仅使用一个（或一小批）样本
training epoch	训练轮次	遍历完一遍全部训练数据
online gradient descent	在线梯度下降	适用于数据流（stream）场景，数据来一个更新一次

Algorithm 1 Stochastic gradient descent

Input: Training set of data points indexed by $n \in \{1, \dots, N\}$

Error function per data point $E_n(\mathbf{w})$

Learning rate parameter η

Initial weight vector $\mathbf{w}$

Output: Final weight vector $\mathbf{w}$

$n \leftarrow 1$

repeat

$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla E_n(\mathbf{w})$

// update weight vector

$n \leftarrow n + 1 \pmod{N}$

// iterate over data

until convergence

return $\mathbf{w}$

9 P9

A further advantage of stochastic gradient descent, compared to batch gradient descent, is that it handles redundancy in the data much more efficiently.

To see this, consider an extreme example in which we take a data set and double its size by duplicating every data point.

Note that this simply multiplies the error function by a factor of 2 and so is equivalent to using the original error function, if the value of the learning rate is adjusted to compensate.

Batch methods will require double the computational effort to evaluate the batch error function gradient, whereas stochastic gradient descent will be unaffected.

Another property of stochastic gradient descent is the possibility of escaping from local minima, since a stationary point with respect to the error function for the whole data set will generally not be a stationary point for each data point individually.

翻译:

与批量梯度下降相比，随机梯度下降的另一个优势在于它能更高效地处理数据中的冗余。

为了说明这一点，考虑一个极端例子：假设我们有一个数据集，并通过复制每个数据点使其规模翻倍。

需要注意的是，这仅仅相当于将误差函数乘以 2；因此，只要相应调整学习率加以补偿，就等价于使用原始的误差函数。

在这种情况下，批量方法评估批量误差函数梯度所需的计算量会翻倍，而随机梯度下降则不受影响。

随机梯度下降的另一个特性是它有可能逃离局部极小值。这是因为，对于整个数据集的误差函数而言的驻点，通常并不是单个数据点误差函数的驻点。

问：为什么在数据集复制加倍的情况下，批量方法评估批量误差函数梯度所需的计算量会翻倍，而随机梯度下降则不受影响？

答：这主要源于两种方法在单次参数更新时计算梯度的机制不同。我们可以从数学定义和实际操作两个层面来理解：

1. 批量梯度下降（Batch Gradient Descent）

批量方法的每一次权重更新，都必须计算整个数据集上的平均（或总和）梯度。

原始数据集：包含 N 个样本，计算一次梯度需要遍历 N 个点，计算量为 $O(N)$ 。

翻倍后的数据集：包含 $2N$ 个样本（每个点重复了一次）。虽然数学上 $\nabla E_{new} = 2\nabla E_{old}$ ，但算法在执行时并不知道这一点，它仍然会老实地遍历所有 $2N$ 个样本进行累加。

结果：为了得到这一个更新方向的梯度，CPU/GPU 必须执行两倍的加法/乘法运算。计算量严格翻倍。

2. 随机梯度下降（SGD）

SGD 的每一次权重更新，只使用一个（或一小批）样本来估计梯度。

更新公式： $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla E_n(\mathbf{w})$

数据冗余的影响：当数据集翻倍（简单复制）后，SGD 在采样时，抽到某个原始样本或其副本的概率确实变了，但单次更新所涉及的计算量始终是处理 1 个样本。

结果：无论数据集里有 100 个点还是 100 万个重复点，SGD 迈出“一步”所需的浮点运算次数是完全一样的。因此，单步计算成本不受数据集大小影响。

核心区别总结：

维度	批量方法	随机梯度下降 (SGD)
单步梯度计算对象	全部 N 个样本	1 个样本（第 n 个样本）
数据翻倍后的单步计算量	$\times 2$ （因为 $N \rightarrow 2N$ ）	$\times 1$ （不变，仍是 1 个样本）
对冗余的敏感度	高（冗余直接转化为无效计算）	低（冗余仅改变采样概率，不增加单步开销）

重要补充说明：虽然 SGD 的单步计算量不受影响，但这并不意味着“训练总时间”完全不变。数据翻倍后，如果保持相同的遍历次数（Epochs），SGD 需要采样的总步数也会翻倍才能看完一遍数据。

原文强调“不受影响”，特指“评估梯度以完成一次参数更新”这一原子操作的效率。这正是 SGD 适合大数据集的根本原因：它用极低的单步成本换取了快速的迭代，而不必像批量方法那样，每走一步都要付出与数据总量成正比的沉重代价。

10 P10

A downside of stochastic gradient descent is that the gradient of the error function computed from a single data point provides a very noisy estimate of the gradient of the error function computed on the full data set.

We can consider an intermediate approach in which a small subset of data points, called a *mini-batch*, is used to evaluate the gradient at each iteration.

In determining the optimum size for the mini-batch, note that the error in computing the mean from N samples is given by $\sigma/\sqrt{N}$ where σ is the standard deviation of the distribution generating the data.

This indicates that there are diminishing returns in estimating the true gradient from increasing the batch size.

If we increase the size of the mini-batch by a factor of 100 then the error only reduces by a factor of 10.

Another consideration in choosing the mini-batch size is the desire to make efficient use of the hardware architecture on which the code is running.

For example, on some hardware platforms, mini-batch sizes that are powers of 2 (for example, 64, 128, 256, ...) work well.

翻译:

随机梯度下降的一个缺点是，基于单个数据点计算出的误差函数梯度，是对全数据集误差函数梯度的一个噪声极大的估计。

我们可以考虑一种折中方案：在每次迭代中，使用一小部分数据点（称为“小批量”）来评估梯度。

在确定小批量的最优大小时，需要注意：基于 N 个样本计算均值时的误差为 $\sigma/\sqrt{N}$ ，其中 σ 是生成数据的分布的标准差。

这表明，通过增大批量大小来提高真实梯度估计的精度，其收益是递减的。

例如，若将小批量大小增大 100 倍，误差仅缩小为原来的十分之一。

选择小批量大小时还需考虑的另一个因素是，要充分利用代码所运行的硬件架构。

例如，在某些硬件平台上，取 2 的幂次作为小批量大小（如 64、128、256 等）往往能获得较好的性能表现。

11 P11

One important consideration when using mini-batches is that the constituent data points should be chosen randomly from the data set, since in raw data sets there may be correlations between successive data points arising from the way the data was collected (for example, if the data points have been ordered alphabetically or by date).

This is often handled by randomly shuffling the entire data set and then subsequently drawing mini-batches as successive blocks of data.

The data set can also be reshuffled between iterations through the data set, so that each mini-batch is unlikely to have been used before, which can help escape local minima.

The variant of stochastic gradient descent with mini-batches is summarized in Algorithm 7.2.

Note that the learning algorithm is often still called ‘stochastic gradient descent’ even when mini-batches are used.

使用小批量时的一个重要注意事项是，其中的数据点应当从数据集中随机选取。这是因为原始数据集中，相邻数据点之间可能存在由数据采集方式引入的相关性（例如，数据点可能已按字母顺序或日期排序）。

通常的做法是先对整个数据集进行随机打乱，然后依次抽取连续的数据块作为小批量。

在遍历数据集的过程中，还可以在各轮迭代之间重新打乱数据，使得每个小批量都不太可能与之前使用过的完全相同，这有助于逃离局部极小值。

带小批量的随机梯度下降变体总结于算法 7.2 中。

需要注意的是，即使使用了小批量，该学习算法通常仍被称为“随机梯度下降”。

Algorithm 2 Mini-batch stochastic gradient descent

Input: Training set of data points indexed by $n \in \{1, \dots, N\}$

Batch size B

Error function per mini-batch $E_{n:n+B-1}(\mathbf{w})$

Learning rate parameter η

Initial weight vector $\mathbf{w}$

Output: Final weight vector $\mathbf{w}$

$n \leftarrow 1$

repeat

$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla E_{n:n+B-1}(\mathbf{w})$

 // update weight vector

$n \leftarrow n + B$

 // next mini-batch

if $n > N$ **then**

 shuffle data

 // 如已遍历数据集，则重组小批量

$n \leftarrow 1$

end if

until convergence

return $\mathbf{w}$

12 P12

Iterative algorithms such as gradient descent require that we choose some initial setting for the parameters being learned.

The specific initialization can have a significant effect on how long it takes to reach a solution and on

the generalization performance of the resulting trained network.

Unfortunately, there is relatively little theory to guide the initialization strategy.

One key consideration, however, is *symmetry breaking*.

Consider a set of hidden units or output units that take the same inputs.

If the parameters were all initialized with the same value, for example if they were all set to zero, the parameters of these units would all be updated in unison and the units would each compute the same function and hence be redundant.

This problem can be addressed by initializing parameters randomly from some distribution to break symmetry.

If computational resources permit, the network might be trained multiple times starting from different random initializations and the results compared on held-out data.

翻译:

像梯度下降这样的迭代算法，要求我们为待学习的参数选择某种初始设置。

具体的初始化方式会显著影响算法收敛所需的时间，以及最终训练所得网络的泛化性能。

遗憾的是，目前关于初始化策略的理论指导相对较少。

然而，其中一个关键考量是对称性破缺（symmetry breaking）。

考虑一组接收相同输入的隐藏层单元或输出层单元。如果它们的参数全部被初始化为相同的值（例如，全部设为零），那么这些单元的参数在更新时也会完全同步，导致每个单元计算出的函数完全相同，从而变得完全冗余。

这一问题可以通过从某种分布中随机初始化参数来解决，以此来打破对称性。

如果计算资源允许，可以从不同的随机初始点出发多次训练网络，并在留出的测试数据（held-out data）上比较它们的结果。

问：考虑一组接收相同输入的隐藏层单元或输出层单元。如果它们的参数全部被初始化为相同的值（例如，全部设为零），那么这些单元的参数在更新时也会完全同步，导致每个单元计算出的函数完全相同，从而变得完全冗余。这句话在说什么？

答：这句话是在解释神经网络训练中一个非常经典的陷阱——“对称性陷阱”。

为了让你更直观地理解，我们可以把这句话拆解成三个部分，并配合一个通俗的比喻：

1. 为什么“接收相同输入”会导致问题？

在神经网络中，同一层的神经元通常是“并联”的，它们接收的是上一层完全相同的输入信号。

2. 如果“初始值相同”（比如全是 0），会发生什么？

假设第 2 层有两个神经元（神经元 A 和神经元 B），它们的权重初始值全都是 0。

前向传播时：因为输入相同、权重相同，它们计算出的输出结果必然完全一模一样。

反向传播时：因为输出相同，它们对误差的“贡献”也一模一样，所以反向传播算出来的梯度（即参数更新的步长）也必然完全一模一样。

更新后：既然初始值相同，增加的梯度也相同，那么更新后的权重依然完全相同。

3. 最终后果：“变得完全冗余”

无论你训练多少个回合（Epoch），神经元 A 和神经元 B 永远像“连体婴儿”一样，步调完全一致，计算出的特征也完全相同。

这就意味着，你明明设置了两个神经元，但它们在数学上等价于一个神经元。你白白浪费了计算资源，网络的表达能力（容量）并没有因为增加神经元而提升。

一个通俗的比喻：

想象你有一个团队，要派两个员工（A 和 B）去执行相同的任务。

如果一开始给他们分配的任务方向完全一样（初始化为 0），他们每天收到的反馈也一样，那么他们永远只会做一模一样的事情，变成互相的“复制粘贴”。

解决办法（对称性破缺）：一开始就给他们随机分配不同的初始方向（随机初始化），这样他们就会各自探索不同的路径，学到不同的东西，团队才有意义。

总结来说：这句话的核心意思就是——在神经网络中，同一层的权重绝对不能初始化为相同的值（尤其是不能全初始化为 0），必须引入随机性，让每个神经元从一开始就“与众不同”，这样它们才能学到不同的特征。

不过这里有个反直觉的地方：虽然不能全初始化为 0，但也不能初始化得太大。这是为什么呢？

13 P13

The distribution used to initialize the weights is typically either a uniform distribution in the range $[-\epsilon, \epsilon]$ or a zero-mean Gaussian of the form $\mathcal{N}(0, \epsilon^2)$.

The choice of the value of ϵ is important, and various heuristics to select it have been proposed.

One widely used approach is called *He initialization* (He et al., 2015b).

Consider a network in which layer l evaluates the following transformations

$$a_i^{(l)} = \sum_{j=1}^M w_{ij} z_j^{(l-1)} \quad (7.19)$$

$$z_i^{(l)} = \text{ReLU}(a_i^{(l)}) \quad (7.20)$$

where M is the number of units that send connections to unit i , and the ReLU activation function is given by (6.17).

Suppose we initialize the weights using a Gaussian $\mathcal{N}(0, \epsilon^2)$, and suppose that the outputs $z_j^{(l-1)}$ of the units in layer $l - 1$ have variance λ^2 .

Then we can easily show that

$$\mathbb{E}[a_i^{(l)}] = 0 \quad (7.21)$$

$$\text{var}[z_j^{(l)}] = \frac{M}{2} \epsilon^2 \lambda^2 \quad (7.22)$$

where the factor of $1/2$ arises from the ReLU activation function.

Ideally we want to ensure that the variance of the pre-activations neither decays to zero nor grows significantly as we propagate from one layer to the next.

If we therefore require that the units at layer l also have variance λ^2 then we arrive at the following choice for the standard deviation of the Gaussian used to initialize the weights that feed into a unit

with M inputs:

$$\epsilon = \sqrt{\frac{2}{M}}. \quad (7.23)$$

翻译:

用于初始化权重的分布，通常是在 $[-\epsilon, \epsilon]$ 范围内的均匀分布，或者是形如 $\mathcal{N}(0, \epsilon^2)$ 的零均值高斯分布。

ϵ 的取值选择非常重要，为此人们提出了各种启发式方法。

其中一种被广泛使用的方法被称为 He 初始化 (He 等, 2015b)。

考虑一个网络，其第 ℓ 层执行如下变换：

$$a_i^{(\ell)} = \sum_{j=1}^M w_{ij} z_j^{(\ell-1)} \quad (7.19)$$

$$z_i^{(\ell)} = \text{ReLU}(a_i^{(\ell)}) \quad (7.20)$$

其中 M 是向单元 i 发送连接的单元数量，ReLU 激活函数由式 (6.17) 给出。

假设我们使用高斯分布 $\mathcal{N}(0, \epsilon^2)$ 来初始化权重，并且假设第 $\ell-1$ 层各单元的输出 $z_j^{(\ell-1)}$ 的方差为 λ^2 。

那么我们可以很容易地证明：

$$\mathbb{E}[a_i^{(\ell)}] = 0 \quad (7.21)$$

$$\text{var}[z_j^{(\ell)}] = \frac{M}{2} \epsilon^2 \lambda^2 \quad (7.22)$$

其中因子 $1/2$ 正是由 ReLU 激活函数引起的。

理想情况下，我们希望确保在逐层传播的过程中，激活前 (pre-activations) 的方差既不会衰减至零，也不会显著增大。

因此，如果我们要求第 ℓ 层的单元输出也具有 λ^2 的方差，那么对于具有 M 个输入的单元，用于初始化其输入权重的高斯分布的标准差应选取为：

$$\epsilon = \sqrt{\frac{2}{M}}. \quad (7.23)$$

要点提示:

这段话解释了为什么 He 初始化公式里有个“2”。

因为 ReLU 激活函数会将一半的负数输出直接截断为 0。这会导致数据的方差在每次经过 ReLU 后损失一半（即公式 7.22 中的 $1/2$ ）。为了弥补这个损失，让信号在深层网络中传播时方差保持稳定，He 初始化特意在标准差公式中乘上了 $\sqrt{2}$ （即公式 7.23），以抵消 ReLU 带来的方差衰减。

公式 (7.21)(7.22) 要用到根据 w_{ij} 与 $z_j^{(\ell-1)}$ 相互独立，且均值都是零。

14 P14

It is also possible to treat the scale ϵ of the initialization distribution as a hyperparameter and to explore different values across multiple training runs.

The bias parameters are typically set to small positive values to ensure that most pre-activations are initially active during learning.

This is particularly helpful with ReLU units, where we want the pre-activations to be positive so that there is a non-zero gradient to drive learning.

Another important class of techniques for initializing the parameters of a neural network is by using the values that result from training the network on a different task or by exploiting various forms of unsupervised training.

These techniques fall into the broad class of *transfer learning* techniques.

翻译：

也可以将初始化分布的尺度 ϵ 视为一个超参数，并在多次训练运行中尝试不同的取值。

偏置参数通常被初始化为较小的正值，以确保在学习初期，大多数激活前（pre-activations）处于激活状态。

这对于 ReLU 单元尤为有用，因为我们希望激活前保持为正值，从而产生非零的梯度来驱动学习。

初始化神经网络参数的另一类重要技术，是利用网络在其他任务上训练所得的参数值，或者利用各种形式的无监督训练得到的参数值。

这些技术属于广义上的迁移学习（transfer learning）技术范畴。

问：什么是超参数？

答：超参数（Hyperparameter）是机器学习和神经网络中一个非常核心的概念。

简单来说，超参数是“用来控制模型学习过程的参数”，而不是模型自己“学”出来的参数。

为了让你彻底弄懂，我们可以从以下几个维度来理解：

1. 超参数 vs 参数（Parameters）

在神经网络中，我们需要严格区分这两个词：

参数（Parameters）：也就是模型内部的权重（Weights）和偏置（Biases）。它们是模型在训练过程中，通过反向传播算法和数据自动学习/更新得到的。

超参数（Hyperparameters）：是在训练开始之前，由人类（工程师）根据经验或实验手动设定的。在训练过程中，它们通常是保持不变的。

2. 常见的超参数有哪些？在你刚才阅读和翻译的文本中，就提到了一个非常经典的超参数。除了它，还有几个常见的例子：

学习率（Learning rate, η ）：决定每次参数更新时“迈多大的步子”。

初始化分布的尺度（Scale ϵ ）：也就是你刚才翻译的那段话中提到的，决定初始权重数值大小的范围。

小批量大小（Mini-batch size）：每次更新权重时，使用多少个样本。

网络结构：比如隐藏层有几层、每层有多少个神经元。

正则化系数：用来防止模型过拟合的惩罚项权重。

3. 为什么叫“超”参数？

你可以把它理解为“站在更高层次的参数”。

普通的参数（权重）是模型在“做题”（前向传播和反向传播）时自己调整的；而超参数是“教练”在“布置训练计划”时设定的规则。教练的规则（超参数）决定了模型做题的效率，但教练的规则本身不会在做题过程中被改变。

4. 超参数是怎么确定的？

既然超参数是人定的，那定多少合适呢？这就引出了机器学习中常说的“调参（Hyperparameter tuning）”。

就像你刚才翻译的那段话所说：“将初始化分布的尺度 ϵ 视为一个超参数，并在多次训练运行中尝试不同的取值”。在实际工程中，我们通常会尝试一系列不同的超参数组合，跑多次训练，最后看哪一组超参数能让模型在验证集上的表现最好。
